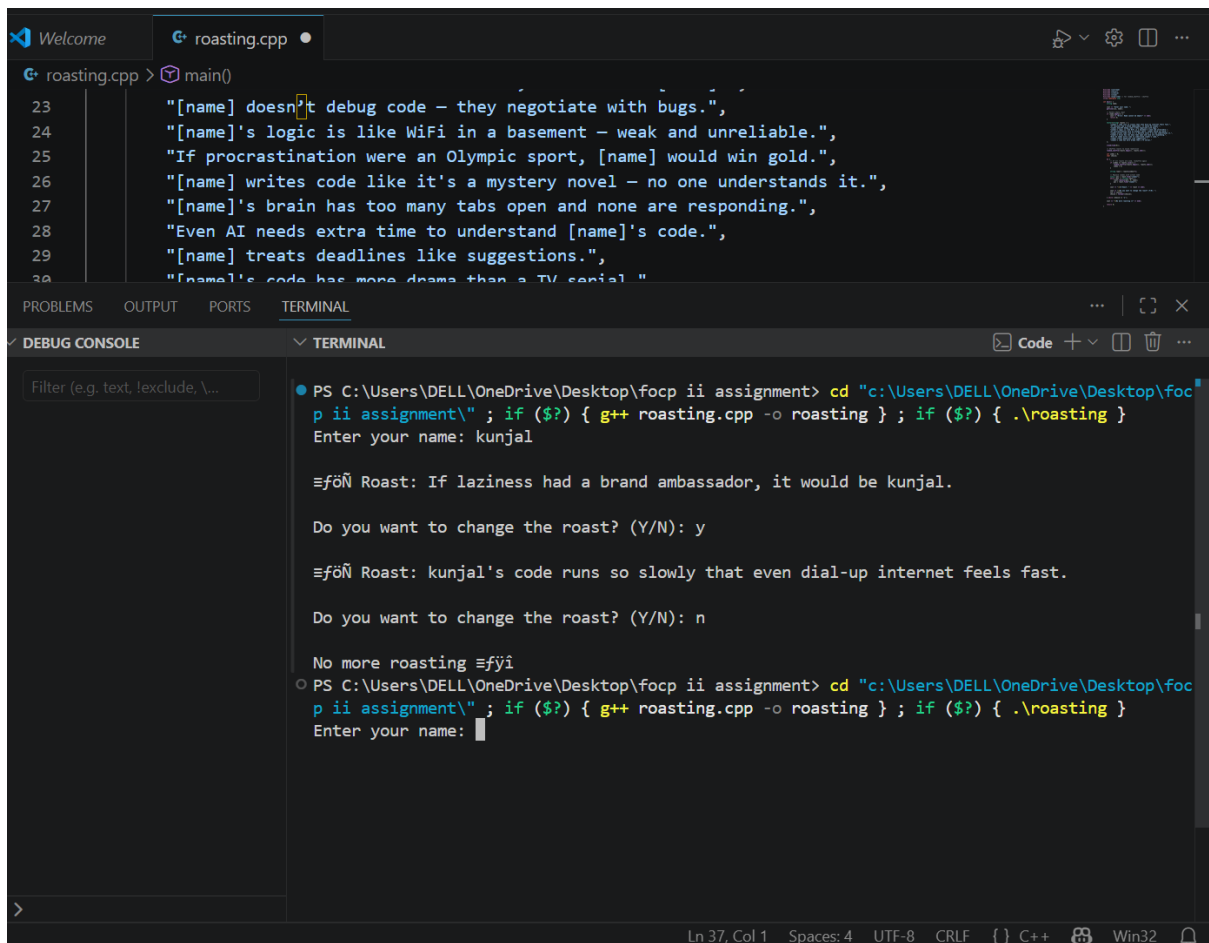


output of the code



The screenshot shows a C++ IDE with a file named `roasting.cpp`. The code in the editor contains several roast templates for a person named [name]. The terminal window shows the execution of the program. The user enters their name as 'kunjai'. The program outputs a roast: 'Roast: If laziness had a brand ambassador, it would be kunjai.' The user is then asked if they want to change the roast. They enter 'y'. The program outputs another roast: 'Roast: kunjai's code runs so slowly that even dial-up internet feels fast.' The user is asked again if they want to change the roast. They enter 'n'. The program then outputs 'No more roasting' and ends.

```
roasting.cpp > main()
23     "[name] doesn't debug code - they negotiate with bugs.",
24     "[name]'s logic is like WiFi in a basement - weak and unreliable.",
25     "If procrastination were an Olympic sport, [name] would win gold.",
26     "[name] writes code like it's a mystery novel - no one understands it.",
27     "[name]'s brain has too many tabs open and none are responding.",
28     "Even AI needs extra time to understand [name]'s code.",
29     "[name] treats deadlines like suggestions.",
30     "[name]'s code has more drama than a TV serial "

PROBLEMS OUTPUT PORTS TERMINAL
DEBUG CONSOLE TERMINAL
Filter (e.g. text, lexclude, \...
• PS C:\Users\DELL\OneDrive\Desktop\foep ii assignment> cd "c:\Users\DELL\OneDrive\Desktop\foep ii assignment\" ; if ($?) { g++ roasting.cpp -o roasting } ; if ($?) { .\roasting }
Enter your name: kunjai

≡föÑ Roast: If laziness had a brand ambassador, it would be kunjai.

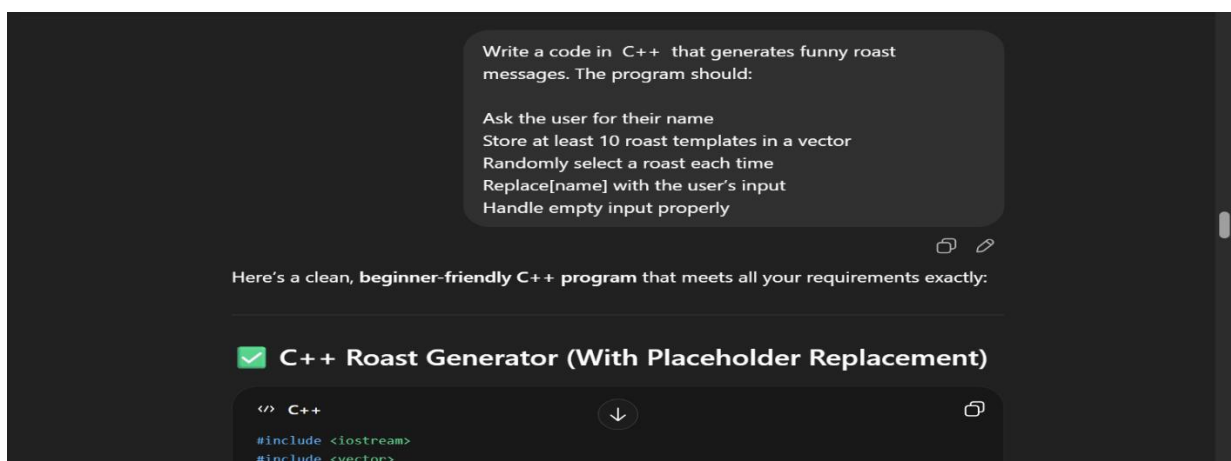
Do you want to change the roast? (Y/N): y

≡föÑ Roast: kunjai's code runs so slowly that even dial-up internet feels fast.

Do you want to change the roast? (Y/N): n

No more roasting ≡fÿi
• PS C:\Users\DELL\OneDrive\Desktop\foep ii assignment> cd "c:\Users\DELL\OneDrive\Desktop\foep ii assignment\" ; if ($?) { g++ roasting.cpp -o roasting } ; if ($?) { .\roasting }
Enter your name: 
```

Prompt for roasting



The screenshot shows a chat interface with a prompt for a C++ roast generator. The prompt asks for a C++ program that generates funny roast messages. The program should ask the user for their name, store at least 10 roast templates in a vector, randomly select a roast each time, replace [name] with the user's input, and handle empty input properly. The chat interface shows a response from the AI, which is a C++ program titled "C++ Roast Generator (With Placeholder Replacement)". The program includes the necessary headers and starts with the include statements.

Write a code in C++ that generates funny roast messages. The program should:

- Ask the user for their name
- Store at least 10 roast templates in a vector
- Randomly select a roast each time
- Replace[name] with the user's input
- Handle empty input properly

Here's a clean, beginner-friendly C++ program that meets all your requirements exactly:

✓ C++ Roast Generator (With Placeholder Replacement)

```
</ C++
#include <iostream>
#include <vector>
```

Provided code

```
#include <iostream>

#include <vector>

#include <cstdlib>

#include <ctime>

using namespace std;

int main() {

    string name;


    // Ask user for name

    cout << "Enter your name: ";

    getline(cin, name);


    // Handle empty input

    if (name.empty()) {

        cout << "Error: Name cannot be empty!" << endl;

        return 0;

    }


    // Roast templates with [name] placeholder

    vector<string> roasts = {

        "[name]'s code runs so slowly that even dial-up internet feels fast.",

        "If laziness had a brand ambassador, it would be [name].",

        "[name] doesn't debug code — they negotiate with bugs.",

        "[name]'s logic is like WiFi in a basement — weak and unreliable.",

        "If procrastination were an Olympic sport, [name] would win gold.",

        "[name] writes code like it's a mystery novel — no one understands it.",
```

```
"[name]'s brain has too many tabs open and none are responding.",
"Even AI needs extra time to understand [name]'s code.",
"[name] treats deadlines like suggestions.",
"[name]'s code has more drama than a TV serial."
};

// Initialize random seed
srand(time(0));

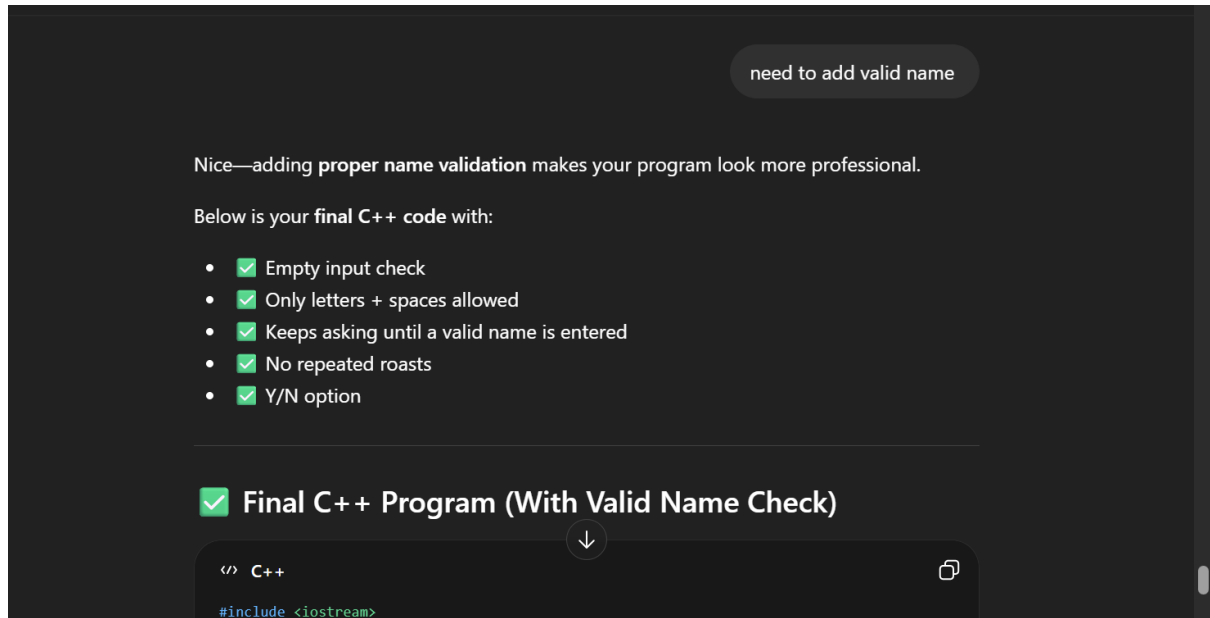
// Pick random roast
int index = rand() % roasts.size();
string roast = roasts[index];

// Replace [name] with actual input
size_t pos = roast.find("[name]");
while (pos != string::npos) {
    roast.replace(pos, 6, name);
    pos = roast.find("[name]");
}

// Display result
cout << "\n 🔥 Roast: " << roast << endl;

return 0;
}
```

Valid name bug



Provided code

```
#include <iostream>  
  
#include <vector>  
  
#include <cstdlib>  
  
#include <ctime>  
  
#include <algorithm>  
  
#include <cctype>  
  
using namespace std;
```

```
// Function to check if name is valid (only letters & spaces)
```

```
bool isValidName(string name) {  
    for (char c : name) {  
        if (!isalpha(c) && c != ' ') {  
            return false;  
        }  
    }  
}
```

```
    }  
}  
return !name.empty();  
}
```

```
int main() {  
    string name;  
  
    // Keep asking until valid name is entered  
    do {  
        cout << "Enter your name: ";  
        getline(cin, name);  
  
        if (!isValidName(name)) {  
            cout << "Invalid name! Use only letters and spaces.\n";  
        }  
    }
```

```
} while (!isValidName(name));
```

```
vector<string> roasts = {  
    "[name]'s code runs so slowly that even dial-up internet feels fast.",  
    "If laziness had a brand ambassador, it would be [name].",  
    "[name] doesn't debug code — they negotiate with bugs.",  
    "[name]'s logic is like WiFi in a basement — weak and unreliable.",  
    "If procrastination were an Olympic sport, [name] would win gold.",  
    "[name] writes code like it's a mystery novel — no one understands it.",  
    "[name]'s brain has too many tabs open and none are responding.",  
    "Even AI needs extra time to understand [name]'s code.",
```

```
"[name] treats deadlines like suggestions.",  
"[name]'s code has more drama than a TV serial."  
};
```

```
srand(time(0));
```

```
// Shuffle to prevent repetition  
random_shuffle(roasts.begin(), roasts.end());
```

```
int index = 0;  
char choice;
```

```
do {  
    if (index == roasts.size()) {  
        random_shuffle(roasts.begin(), roasts.end());  
        index = 0;  
    }  
}
```

```
string roast = roasts[index++];
```

```
// Replace [name]  
size_t pos = roast.find("[name]");  
while (pos != string::npos) {  
    roast.replace(pos, 6, name);  
    pos = roast.find("[name]");  
}
```

```
cout << "\n 🍷 Roast: " << roast << endl;
```

```

    cout << "\nDo you want to change the roast? (Y/N): ";

    cin >> choice;

    choice = tolower(choice);

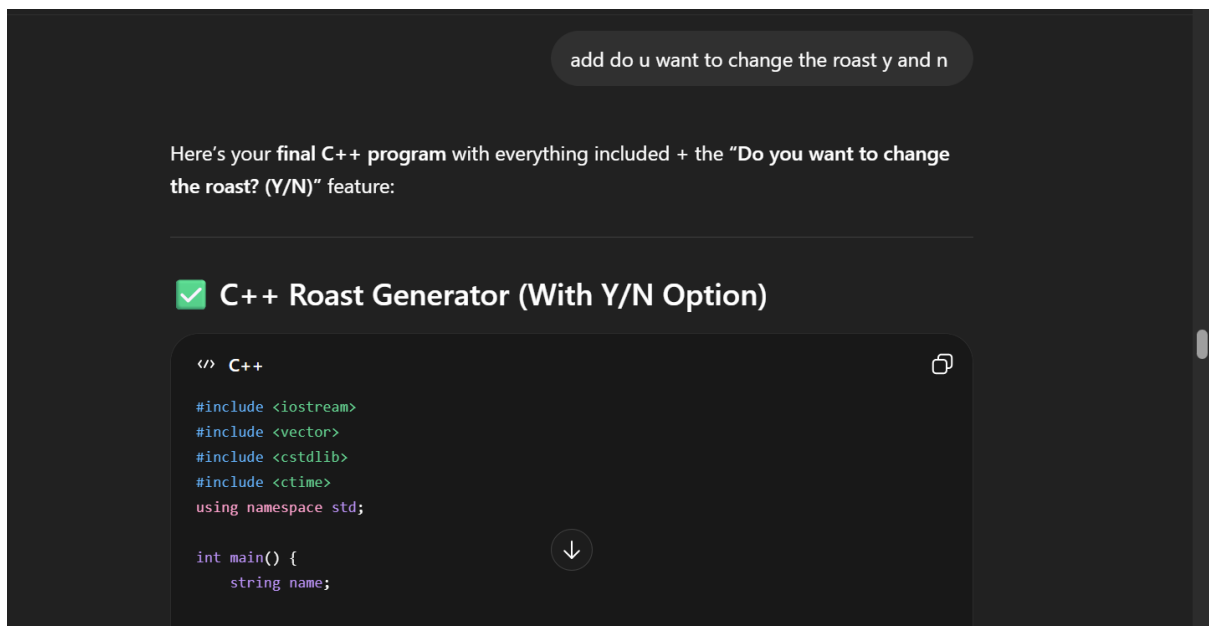
} while (choice == 'y');

cout << "\nNo more roasting 😊" << endl;

return 0;
}

```

add want to change roasting y and n



```

#include <iostream>

#include <vector>

#include <cstdlib>

#include <ctime>

#include <algorithm> // for random_shuffle / shuffle

```

```
using namespace std;
```

```
int main() {
```

```
    string name;
```

```
    cout << "Enter your name: ";
```

```
    getline(cin, name);
```

```
    // Handle empty input
```

```
    if (name.empty()) {
```

```
        cout << "Error: Name cannot be empty!" << endl;
```

```
        return 0;
```

```
    }
```

```
    vector<string> roasts = {
```

```
        "[name]'s code runs so slowly that even dial-up internet feels fast.",
```

```
        "If laziness had a brand ambassador, it would be [name].",
```

```
        "[name] doesn't debug code — they negotiate with bugs.",
```

```
        "[name]'s logic is like WiFi in a basement — weak and unreliable.",
```

```
        "If procrastination were an Olympic sport, [name] would win gold.",
```

```
        "[name] writes code like it's a mystery novel — no one understands it.",
```

```
        "[name]'s brain has too many tabs open and none are responding.",
```

```
        "Even AI needs extra time to understand [name]'s code.",
```

```
        "[name] treats deadlines like suggestions.",
```

```
        "[name]'s code has more drama than a TV serial."
```

```
    };
```

```
    srand(time(0));
```

```
// Shuffle roasts to avoid repetition
random_shuffle(roasts.begin(), roasts.end());

int index = 0;
char choice;

do {
    // If all roasts are used, reshuffle again
    if (index == roasts.size()) {
        random_shuffle(roasts.begin(), roasts.end());
        index = 0;
    }

    string roast = roasts[index++];

    // Replace [name] with actual name
    size_t pos = roast.find("[name]");
    while (pos != string::npos) {
        roast.replace(pos, 6, name);
        pos = roast.find("[name]");
    }

    cout << "\n 🍷 Roast: " << roast << endl;

    cout << "\nDo you want to change the roast? (Y/N): ";
    cin >> choice;
```

```
choice = tolower(choice);
```

```
} while (choice == 'y');
```

```
cout << "\nNo more roasting 😊" << endl;
```

```
return 0;
```

```
}
```

repeat roast

ya prevent same roast repeating

Good call—that's a nice improvement and easy marks.

Here's your final C++ program with NO repeated roasts (until all are used) + Y/N loop:

✓ C++ Roast Generator (No Repeats + Y/N)

```
</> C++  
  
#include <iostream>  
#include <vector>  
#include <cstdlib>  
#include <ctime>  
#include <algorithm> // for random_shuffle / shuffle  
using namespace std;  
  
int main() {  
    string name;
```

Final code

```
#include <iostream>
```

```
#include <vector>
```

```
#include <cstdlib>
```

```
#include <ctime>
```

```
#include <algorithm>
```

```

#include <cctype>

using namespace std;

// Function to check if name is valid (only letters & spaces)
bool isValidName(string name) {
    for (char c : name) {
        if (!isalpha(c) && c != ' ') {
            return false;
        }
    }
    return !name.empty();
}

int main() {
    string name;

    // Keep asking until valid name is entered
    do {
        cout << "Enter your name: ";
        getline(cin, name);

        if (!isValidName(name)) {
            cout << "Invalid name! Use only letters and spaces.\n";
        }

    } while (!isValidName(name));

    vector<string> roasts = {

```

```
"[name]'s code runs so slowly that even dial-up internet feels fast.",  
"If laziness had a brand ambassador, it would be [name].",  
"[name] doesn't debug code — they negotiate with bugs.",  
"[name]'s logic is like WiFi in a basement — weak and unreliable.",  
"If procrastination were an Olympic sport, [name] would win gold.",  
"[name] writes code like it's a mystery novel — no one understands it.",  
"[name]'s brain has too many tabs open and none are responding.",  
"Even AI needs extra time to understand [name]'s code.",  
"[name] treats deadlines like suggestions.",  
"[name]'s code has more drama than a TV serial."
```

```
};
```

```
srand(time(0));
```

```
// Shuffle to prevent repetition
```

```
random_shuffle(roasts.begin(), roasts.end());
```

```
int index = 0;
```

```
char choice;
```

```
do {
```

```
    if (index == roasts.size()) {
```

```
        random_shuffle(roasts.begin(), roasts.end());
```

```
        index = 0;
```

```
    }
```

```
    string roast = roasts[index++];
```

```
// Replace [name]

size_t pos = roast.find("[name]");
while (pos != string::npos) {
    roast.replace(pos, 6, name);
    pos = roast.find("[name]");
}

cout << "\n 🔥 Roast: " << roast << endl;

cout << "\nDo you want to change the roast? (Y/N): ";
cin >> choice;
choice = tolower(choice);

} while (choice == 'y');

cout << "\nNo more roasting 😊 " << endl;

return 0;
}
```